CECY481 – Lab 6

Web Dynamic Testing: Path Traversal and Cookie Security with OWASP ZAP

Student Name: Rayan Kurdi

Student ID: 2440795

Course: CECY481 – Chapter 6

Date: 12 November 2026

1. Environment and Setup

Item	Value
Host / Hypervisor	MacBook (Apple M4) – UTM
Guest OS	Kali Linux (ARM64)
DAST tool	OWASP ZAP 2.17.0 (Manual Explore, proxied Firefox)
Target	Local Flask report service – http://127.0.0.1:5000 (Werkzeug 3.1.9, Python 3.14.8)
Scope	127.0.0.1 only – no external targets scanned

Note: Kali Linux was used instead of the recommended Ubuntu VM because it runs natively on the Apple Silicon host and includes ZAP and Python. The starter project (app.py, requirements.txt, reports/summary.txt and a fake private.txt) was created in ~/CECY481/starter, the virtual environment was set up, and the Flask service was started.

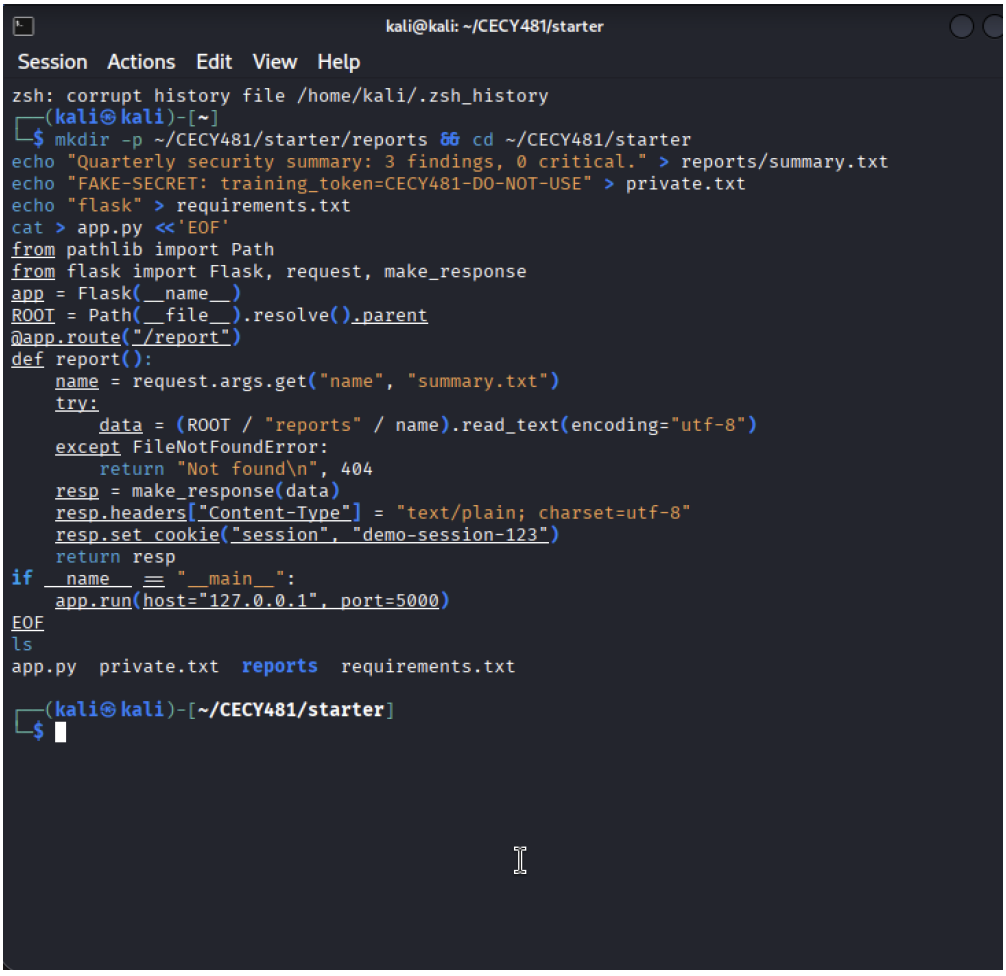


Figure 1: Starter project files created (app.py, private.txt, reports/, requirements.txt)

```
┌──(kali㉿kali)-[~/CECY481/starter]
└─$ python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
Collecting flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker≥1.9.0 (from flask→-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click≥8.1.3 (from flask→-r requirements.txt (line 1))
  Downloading click-8.5.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous≥2.2.0 (from flask→-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2≥3.1.2 (from flask→-r requirements.txt (line 1))
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe≥2.1.1 (from flask→-r requirements.txt (line 1))
  Downloading markupsafe-3.0.4-cp314-cp314-manylinux2014_aarch64.manylinux_2_17_aarch64.ma
ylinux_2_28_aarch64.whl.metadata (2.7 kB)
Collecting werkzeug≥3.1.0 (from flask→-r requirements.txt (line 1))
  Downloading werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.1-py3-none-any.whl (126 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.4-cp314-cp314-manylinux2014_aarch64.manylinux_2_17_aarch64.many
linux_2_28_aarch64.whl (24 kB)
Downloading werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2,
flask
Successfully installed blinker-1.9.0 click-8.5.1 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1
6 markupsafe-3.0.4 werkzeug-3.1.9
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pro
uction WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Figure 2: Virtual environment created, Flask installed and service running on 127.0.0.1:5000

```
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ curl -i "http://127.0.0.1:5000/report?name=summary.txt"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.14.8
Date: Thu, 12 Nov 2026 12:05:09 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 52
Set-Cookie: session=demo-session-123; Path=/
Connection: close

Quarterly security summary: 3 findings, 0 critical.

┌──(kali㉿kali)-[~]
└─$
```

Figure 3: Checkpoint – curl confirms the endpoint returns the sample report (200 OK)

2. A1 – Passive Observation using ZAP (2 marks)

ZAP Quick Start → Manual Explore was used to launch a proxied Firefox to http://127.0.0.1:5000/report?name=summary.txt. The request was recorded in ZAP History (ID 1) and inspected in the Request and Response panes.

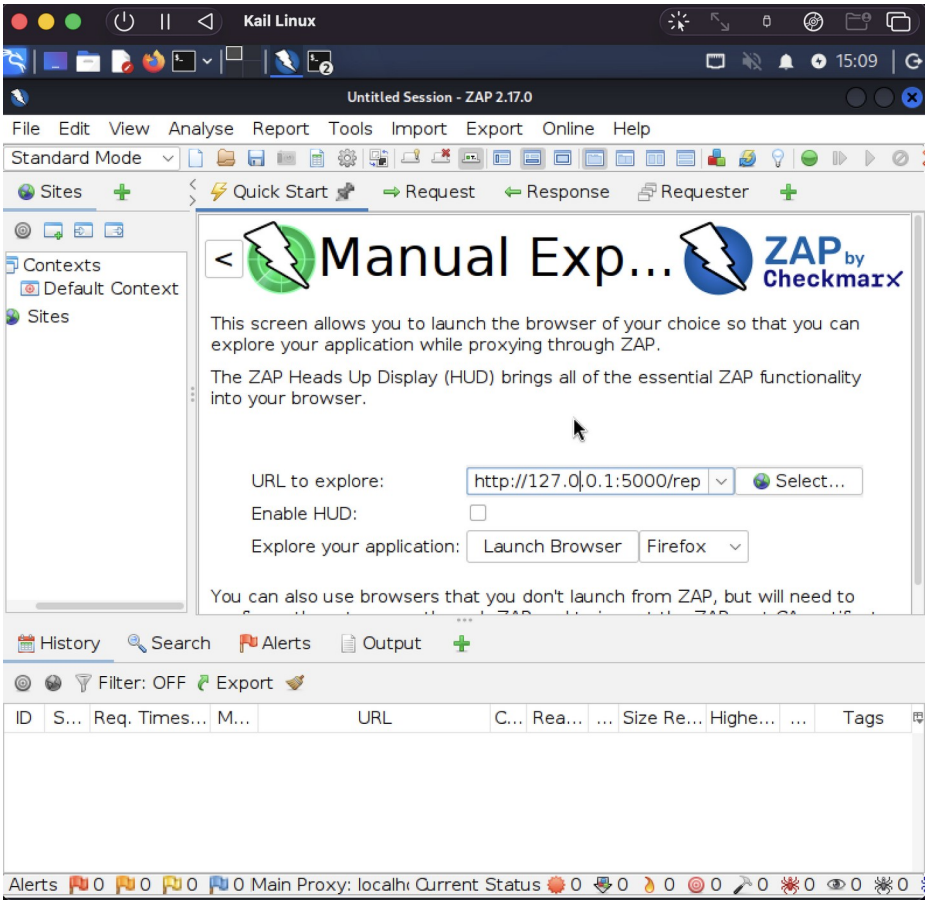


Figure 4: ZAP Manual Explore – launching proxied Firefox

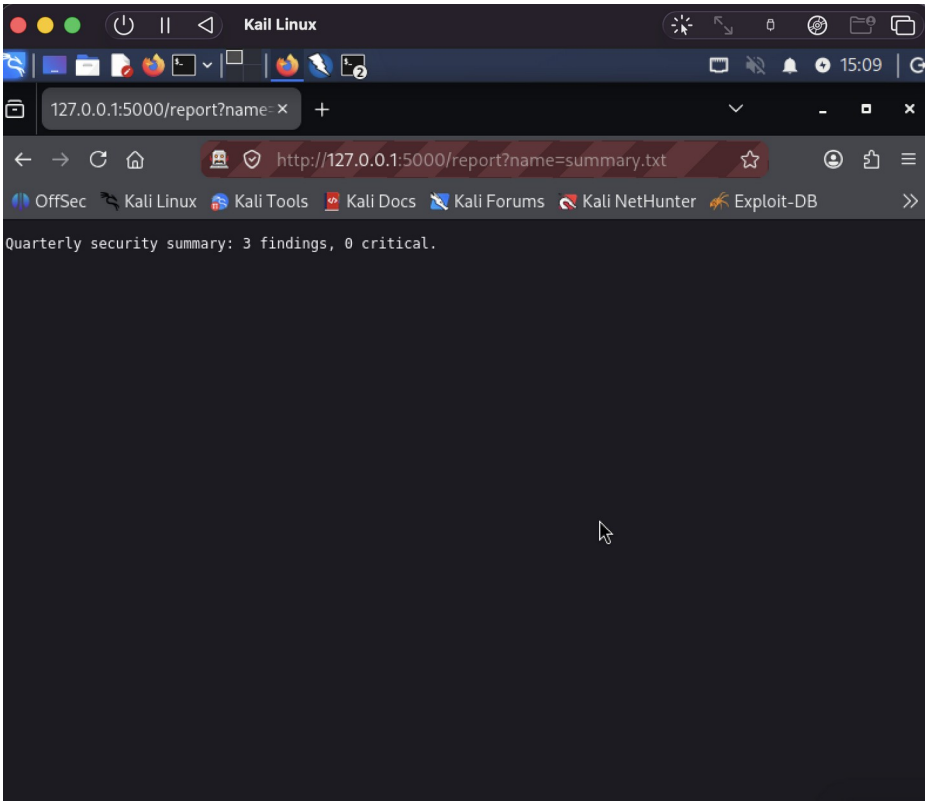


Figure 5: Normal report loaded in the ZAP-launched browser

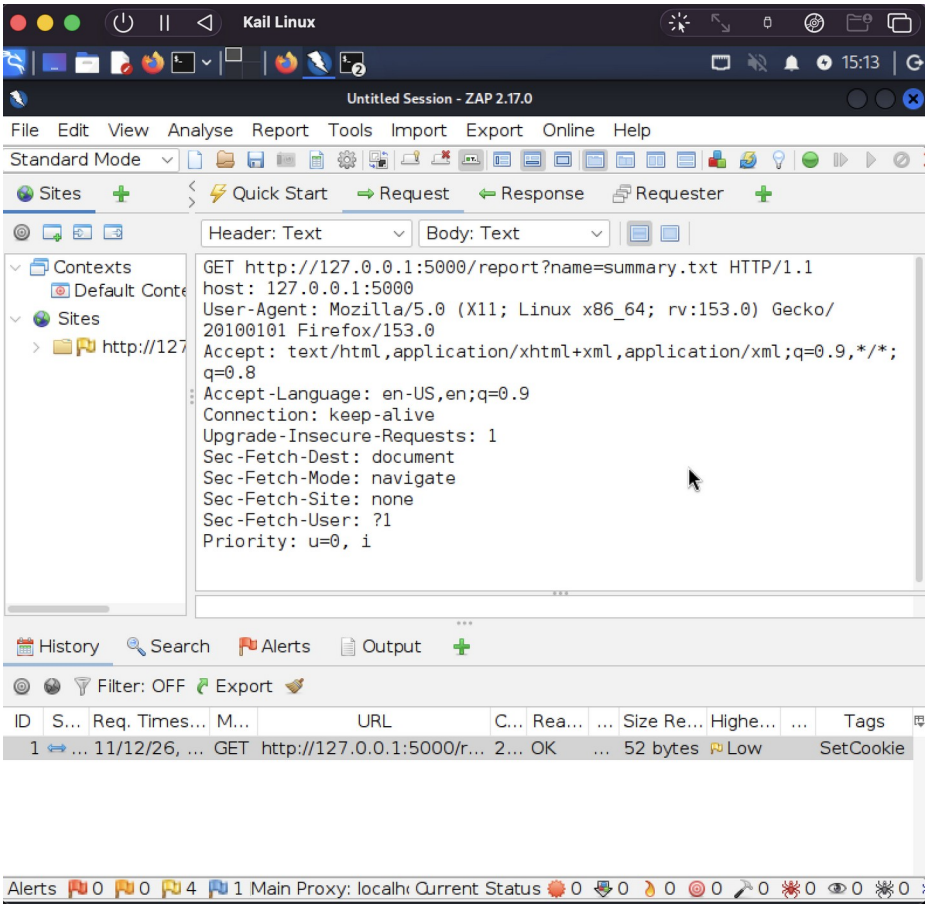


Figure 6: ZAP Request pane – GET /report?name=summary.txt

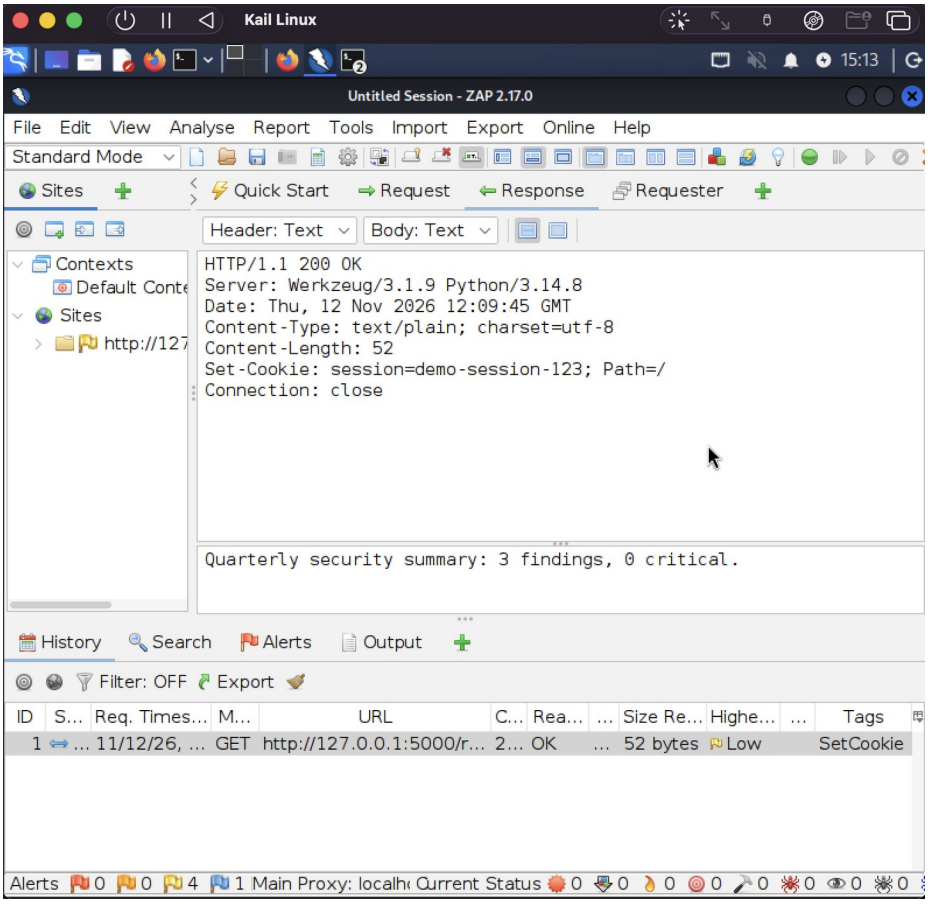


Figure 7: ZAP Response pane – Set-Cookie header with no security attributes

Cookie attribute analysis

```
Set-Cookie: session=demo-session-123; Path=/
```

Attribute	Present?	Risk when missing
Secure	No	Cookie may be sent over plain HTTP and intercepted by a network attacker.
HttpOnly	No	JavaScript can read it via document.cookie, so an XSS flaw could steal the session.
SameSite	No	Browser attaches the cookie to cross-site requests, exposing the app to CSRF.

Why this is passive testing

I only browsed the application normally through the ZAP proxy and observed the traffic it recorded. No input was modified, no payloads were sent and no scan was run, so the server received exactly what a legitimate user would send. Observing normal request/response traffic does not attempt to exploit the application or change its state, so it is passive testing, not an attack.

3. A2 – Active Manual Path Traversal Test (3 marks)

The normal request was repeated, then the crafted request /report?name=../private.txt was sent through the ZAP browser and with curl.

```
curl -i "http://127.0.0.1:5000/report?name=summary.txt"
curl -i "http://127.0.0.1:5000/report?name=../private.txt"
```

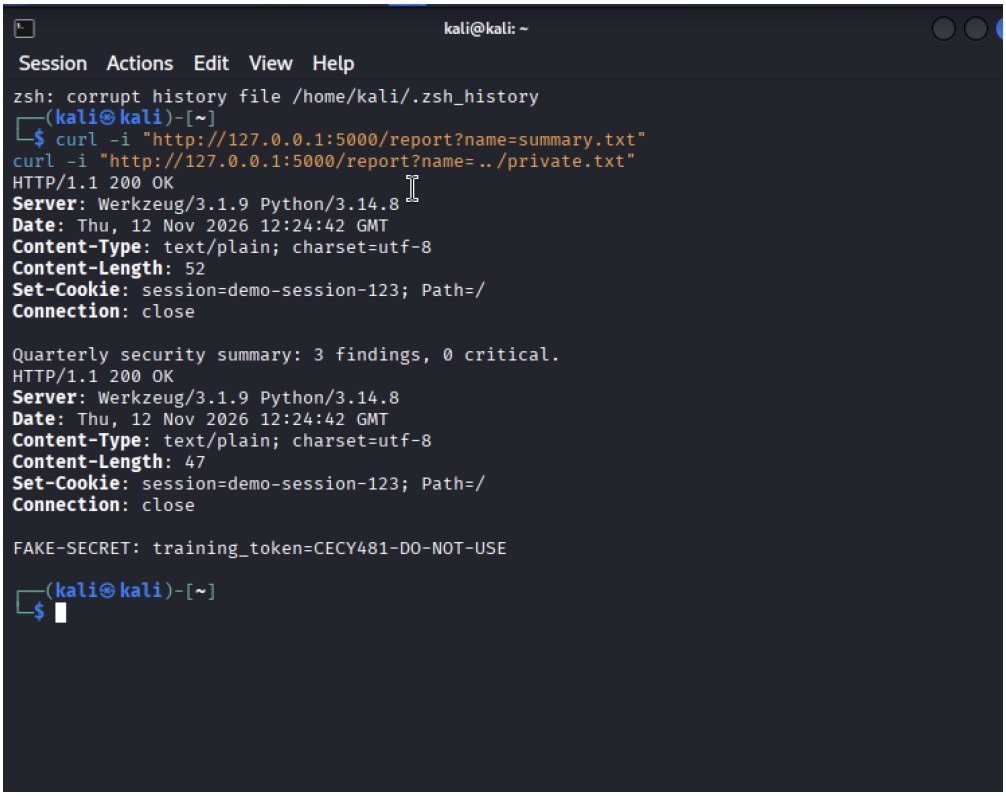


Figure 8: curl – normal request vs. traversal request (both 200 OK)

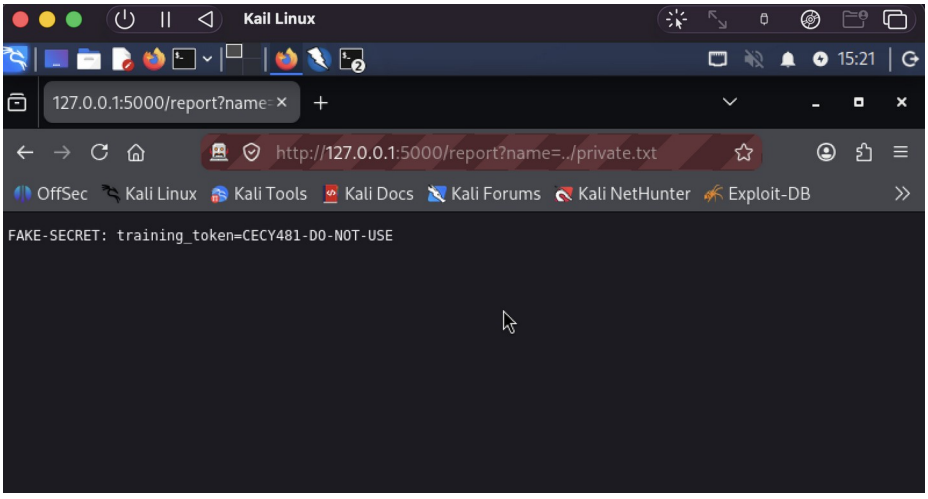


Figure 9: ZAP browser – traversal request returns the contents of private.txt

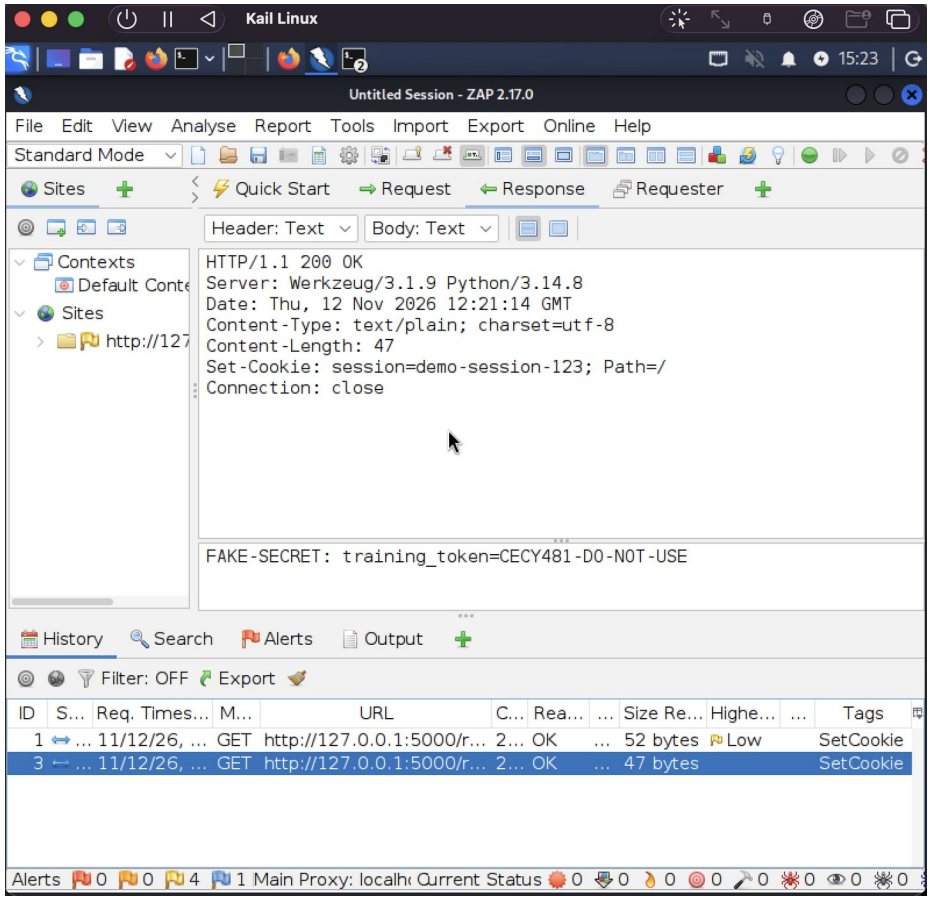


Figure 10: ZAP History (ID 3) – 200 OK response containing private.txt

Request	Status	First line of response body
/report?name=summary.txt	200 OK	Quarterly security summary: 3 findings, 0 critical.
/report?name=../private.txt	200 OK	FAKE-SECRET: training_token=CECY481-DO-NOT-USE

Vulnerability explanation

- **Untrusted input:** the name query parameter, fully controlled by the client.
- **Vulnerable construction:** (ROOT / "reports" / name).read_text(...) joins user input directly into a filesystem path with no validation.
- **Impact:** arbitrary file read (CWE-22) of any file the Flask process can access.

The observed response, not just the payload, confirms the vulnerability: the second request returned status 200 with the contents of private.txt, which sits in the application root outside reports/. The reports/ prefix does not constrain the path, because ../ moves one directory up and reports/../private.txt resolves to private.txt. An attacker could use the same technique to read configuration files, source code or other secrets stored on the server.

4. A3 – DAST Observation and Correlation (2 marks)

ZAP's passive scanner analysed the proxied traffic automatically and raised 5 alerts for the local service. No active scan was run.

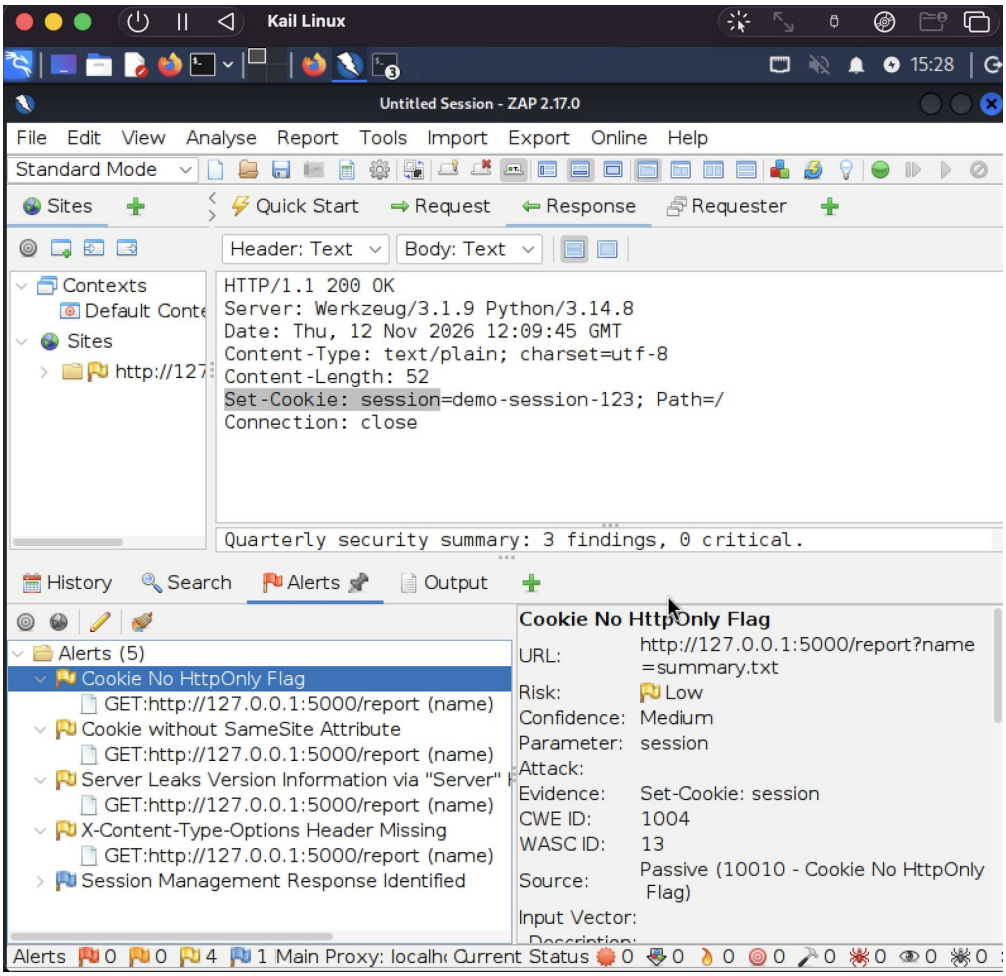


Figure 11: ZAP Alerts – Cookie No HttpOnly Flag details (Risk Low, CWE-1004, Source: Passive)

Alert	Risk	Corroborates
Cookie No HttpOnly Flag	Low	A1
Cookie without SameSite Attribute	Low	A1
Server Leaks Version Information via "Server" header	Low	Additional observation
X-Content-Type-Options Header Missing	Low	Additional observation
Session Management Response Identified	Informational	A1 (session cookie)

Discussion

The two cookie alerts directly corroborate the missing HttpOnly and SameSite attributes observed in A1. ZAP did not report the path traversal from A2, because passive scanning only analyses normal traffic and never sends attack payloads such as ../, so it cannot detect this class of flaw.

An automated alert is a pattern match that shows a weakness is likely, and it can be a false positive or lack context about real exploitability. A manually confirmed vulnerability is proven by sending a specific request and checking the actual response. In A2 that response contained private.txt from outside reports/, which shows both the flaw and its impact. ZAP showed where to look, while manual testing established that the vulnerability exists and can be exploited.

5. A4 – Fix and Verify (3 marks)

Flask was stopped with Ctrl+C, the original file was backed up (app_original.py), and app.py was rewritten with these changes:

- **Allowlist:** only names in ALLOWED_REPORTS = {"summary.txt"} are accepted; anything else returns 403.
- **Containment check (defence in depth):** the path is resolved with .resolve() and must satisfy is_relative_to(REPORTS_DIR); otherwise 403.
- **Secure cookie:** set_cookie(..., secure=True, httponly=True, samesite="Strict").

```
ALLOWED_REPORTS = {"summary.txt"}

def safe_report_path(name: str) -> Path:
    if name not in ALLOWED_REPORTS:
        abort(403, description="Report not allowed")
    candidate = (REPORTS_DIR / name).resolve()
    if not candidate.is_relative_to(REPORTS_DIR):
        abort(403, description="Path outside reports directory")
    if not candidate.is_file():
        abort(404)
    return candidate

resp.set_cookie("session", "demo-session-123",
                secure=True, httponly=True, samesite="Strict")
```

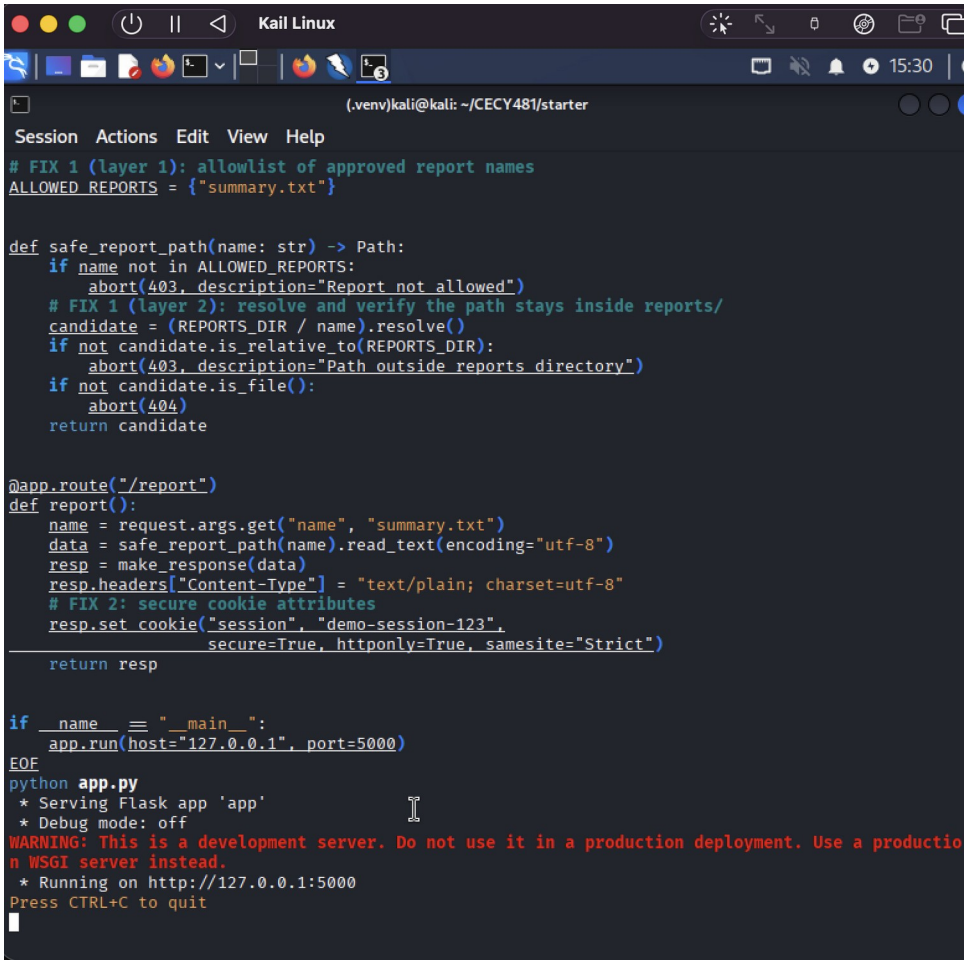


Figure 12: Modified app.py written and Flask restarted

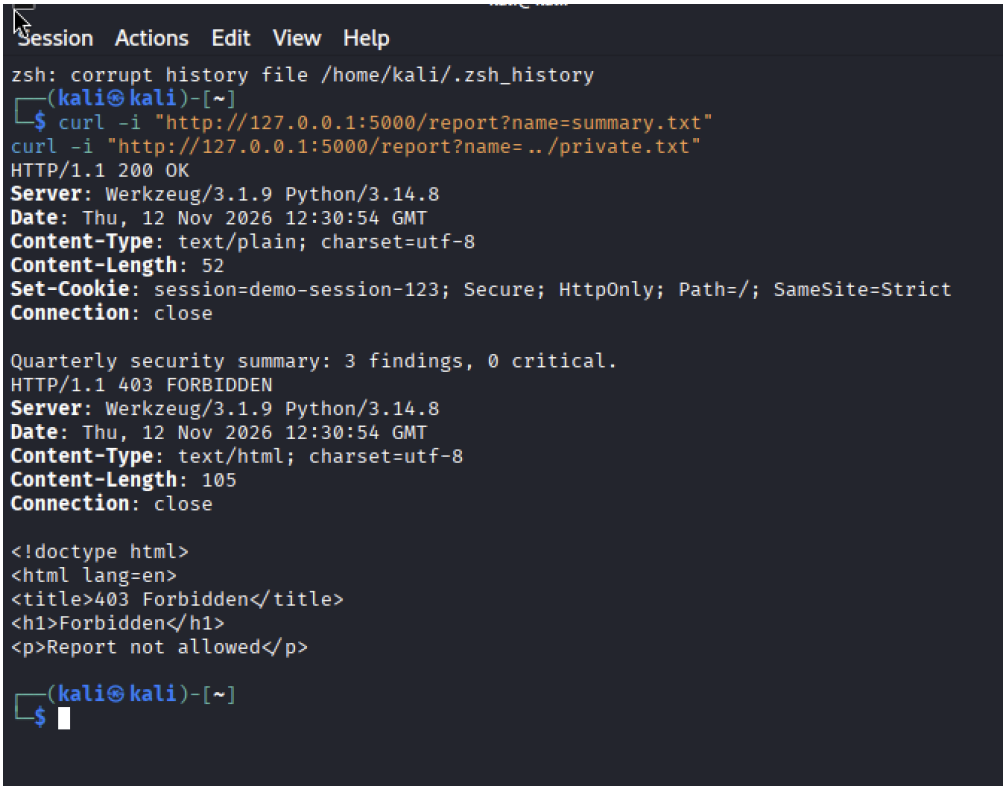


Figure 13: After fix – normal request 200 OK with secure cookie, traversal request 403 Forbidden

Before / after comparison

Request	Before fix	After fix
name=summary.txt	200 OK – report Set-Cookie: session=...; Path=/	200 OK – report Set-Cookie: session=...; Secure; HttpOnly; Path=/; SameSite=Strict
name=../private.txt	200 OK – contents of private.txt	403 FORBIDDEN – "Report not allowed"

Verification statement

After the fix, the authorised use case still works: summary.txt returns 200 with its content. The traversal request is denied with 403 Forbidden. The fix works because user input is no longer treated as a path. It is compared exactly against a fixed list of approved filenames, so sequences such as ../ or absolute paths can never reach the filesystem. The second layer resolves the real path and checks that it stays inside reports/, so a future mistake in the allowlist still could not escape the directory. The session cookie now carries Secure, HttpOnly and SameSite=Strict. Secure only takes effect over HTTPS, so on localhost it was verified from the Set-Cookie response header.

6. Conclusion

Passive observation with ZAP revealed an insecurely configured session cookie, and a manual active test confirmed a path traversal vulnerability that exposed a file outside the intended directory. ZAP's passive alerts corroborated the cookie findings but could not detect the traversal, which shows why manual

verification matters. An allowlist with path containment and secure cookie attributes removed both issues while keeping the legitimate report available.